

Compilation vs. Interpretation

In programming, the way source code is translated into machine-executable instructions determines the performance, portability, and debugging experience of the software.

Feature	Compilation (Compiled)	Interpretation (Interpreted)
Translation Timing	Entire source code is translated at once before execution.	Source code is translated and executed line-by-line during runtime.
Output Product	Generates a standalone executable file (e.g., .exe, .out).	No separate executable is created; the source code is required to run.
Execution Speed	Faster. The translation is done once; the CPU runs pre-optimized machine code.	Slower. The computer must translate each line every time the program runs.
Error Discovery	Errors are reported only after the entire scan is complete. Program won't start if there's a syntax error.	Execution stops the moment an error is reached. Errors are found "on the fly."
Portability	Lower. Compiled code is specific to a hardware architecture (Windows vs. Mac).	Higher. As long as the target machine has the interpreter, the same code can run anywhere.
Typical Languages	C, C++, Rust, Go, Swift.	Python, JavaScript, Ruby, PHP.

Key Takeaway: Compiled languages are built for **efficiency and raw performance**, whereas interpreted languages are built for **flexibility and developer speed**.